Rust KV Scheduling Core Primer 2

Purpose

The project builds a central processing unit (CPU)-side key-value (KV) cache, scheduling, and serving-runtime core for large language model (LLM) inference in Rust. It serves three goals:

1. **Rust Depth:** Learn Rust in depth by building a nontrivial system.
2. **Dynamo Depth:** Learn NVIDIA Dynamo in depth and reach an upstream contribution, in support of the Dynamo Distinguished Engineer posting (JR2009883).
3. **Interview Preparation:** Prepare for the Photon Intelligence coding interview, conducted in Rust or Go with AI coding tools allowed.

Each milestone follows one pattern: Build the component, read Dynamo's counterpart, and compare the two. The build develops Rust skill. The comparison develops Dynamo knowledge and surfaces contribution candidates. The timeboxed core of each milestone serves as an interview drill.

Dynamo already ships Rust implementations of the router, the indexer, the KV Block Manager (KVBM), and the distributed runtime. Each milestone therefore carries a gap extension aimed at a documented Dynamo gap or a Photon technical necessity, which is where contribution value and interview differentiation sit.

Alignment With the Dynamo Posting

Posting Item	Project Coverage
Disaggregated serving	Milestone 7
KV-aware routing	Milestones 3 and 5
Multimodal inference (image, video, audio)	Milestone 1 extension
Resiliency	Milestones 8 and 9
Performance and bottleneck analysis	Performance track, all milestones
Scalability	Milestone 3 concurrency, Milestone 8
Observability and operations	Observability track, all milestones
Storage partner teams	Milestone 10
Open-source contribution	Contribution leads
Rust, Python, and Go	Rust build; Go as interview fallback

Gap Targeting

The Inference Serving Stack Primer records Dynamo's remaining gaps and five technical necessities behind Photon's clean-slate Rust engine. The table assigns each to a milestone.

#	Dynamo Gap Targeted	Photon Necessity Targeted
1	None; foundation	None; foundation
2	Resource Acquisition Is Initialization (RAII) guarantee is single-process and carries no tenant identity	Split KV state
3	None; foundation	Split KV state
4	CPU and graphics processing unit (GPU) overlap, swap versus recompute cost model, speculative decoding remain engine-side	CPU overhead caps GPU utilization
5	Router eligibility as the native fail-closed point	Agentic workload shift
6	Salt is header-derived, guessable, and drops silently	None
7	RAII does not track blocks in transit	Retrofitted disaggregation
8	ZeroMQ (ZMQ) event plane is unauthenticated and exposes block hashes	None
9	Recovery replays tokens and recomputes the KV cache; no deterministic mode	None
10	Lower tiers require explicit point-to-point fetch	Split KV state
Track	No per-tenant cost attribution	None

Status at Current Dynamo Main

The gaps were rechecked against commit `6e4221b` (October 2026). Three findings refine the Inference Serving Stack Primer:

- **Speculative Decoding:** The mocker now models speculative decoding with draft-token counts and acceptance rates (`lib/mocker/src/common/protocols.rs`). Execution remains engine-side.
- **Point-to-Point Block Transfer:** `lib/kvbm-engine/src/leader/session` implements sessions for distributed block transfers, including an RAII `BlockHolder` that holds blocks during a session. The primer lists peer-to-peer KV as a roadmap item; this code supersedes that status in part.
- **Event Plane Authentication:** The NATS transport supports username, token, and NKey authentication and Transport Layer Security (TLS). The ZMQ transport shows no authentication or encryption.

The token-replay migration, the header-derived salt, and the absence of per-tenant cost attribution remain as recorded.

Background: The KV Cache

- To generate each token, the attention layers require a Key vector and a Value vector for every earlier token. Retaining those vectors forms the KV cache.
- The KV cache resides in GPU memory in fixed-size blocks, for example 16 tokens per block, comparable to operating-system memory pages.
- Requests often share opening text, such as a system prompt. When those blocks are already cached, the engine skips recomputing them (prefill), which reduces Time to First Token (TTFT).
- KVBM organizes storage into four tiers: G1 is GPU memory, G2 is CPU pinned memory, G3 is local Non-Volatile Memory Express (NVMe) storage, and G4 is remote storage.
- The milestones cover finding, holding, scheduling, routing, isolating, moving, and recovering reusable blocks.

Milestone Map

#	Component	Dynamo Counterpart	Interview Drill
1	Block hashing	`lib/tokens`, `lib/kv-hashing`	Chained hasher, prefix match
2	Block pool	`lib/kvbm-logical`	Ref-counted allocator, eviction
3	Radix index	`lib/kv-router/src/indexer`	Insert, match, evict
4	Scheduler	`lib/mocker/src/scheduler`	Batching step under a token budget
5	KV router	`lib/kv-router/src/scheduling`	Overlap versus load scoring
6	Tenant keys	`lib/kv-hashing/src/salt.rs`	Keyed salt, fail closed
7	Disaggregated handoff	`lib/kv-router/src/conditional_disagg.rs`, `lib/kvbm-engine/src/leader/session`	Block handoff protocol
8	Discovery and events	`lib/runtime/src/discovery`, `lib/runtime/src/transports`	Lease registry with heartbeat
9	Request migration	`lib/llm/src/migration.rs`	Idempotent retry
10	Offload tier (optional)	`lib/kvbm-physical`, `lib/kvbm-engine`	Two-tier cache

Paths are verified against the Dynamo `main` branch at commit `6e4221b` (October 2026).

Cross-Cutting Tracks

Observability Track

Every milestone emits per-request metrics and traces from its first build: TTFT, Time Per Output Token (TPOT), Inter-Token Latency (ITL), cache hit rate, queue depth, and block utilization. Traces follow one request across router, prefill, and decode once Milestone 7 exists.

- **Gap Extension:** Per-tenant cost attribution, charging prefill tokens, decode tokens, and block-seconds of cache residency to the tenant that incurred them. Dynamo's request trace can capture a tenant header but performs no cost attribution.
- **Rust Concepts:** The `tracing` crate, spans, and metrics exporters.
- **Dynamo Reading:** `lib/llm/src/request_trace`, including the OpenTelemetry sink in `otel_sink.rs`.

Performance Track

Every milestone ends with a benchmark and a written bottleneck finding: What limits throughput or latency, the evidence, and the fix or the reason it was deferred.

- **Rust Concepts:** Criterion benchmarks, flamegraph profiling, allocation counting, and lock contention analysis.
- **Output:** A short findings log per milestone, which also serves as interview material on analyzing complex systems.

Milestone Detail

Milestone 1: Block Hashing

- **Problem:** The engine needs a low-cost test of whether a prompt's opening blocks are already cached.
- **Build:** Split the tokens into blocks and hash each block, mixing in the previous block's hash, so that each hash identifies the full prefix through that block.
- **Example:** Two prompts share their first 32 tokens and then differ. With 16-token blocks, their first two hashes match and the third differs, so the prefix match is two blocks.
- **Multimodal Extension:** Fold image, video, or audio content identifiers into the hash of the block where they occur. Requests carrying the same media at the same position can still share the blocks before it. Dynamo documents this design in `salt.rs`.
- **Rust Concepts:** Ownership, slices, traits, the `Hash` trait, and property-based tests.
- **Dynamo Reading:** `blocks.rs` and `radix.rs` in `lib/tokens`, then `lib/kv-hashing`, which defines the block identity contract shared by the router, KVBM, and the engines.
- **Proving Test:** Hashing a sequence in one pass yields the same hashes as hashing it incrementally while generation appends tokens.

Milestone 2: Block Pool

- **Problem:** GPU memory holds a fixed number of blocks, which must be allocated and reclaimed.
- **Build:** An allocator modeled on the KVBM lifecycle. Blocks move from Reset to Partial as tokens append, then to an inactive pool on commit or eviction, with an active sub-pool and an inactive free list. Reference counts on shared prefix blocks return a block only when its count reaches zero. Finished blocks remain cached until memory pressure triggers least recently used (LRU) eviction. RAII handles return blocks automatically through the `Drop` trait.
- **Gap Extension:** Tenant identity carried on every block handle, so a handle cannot be shared across tenants by construction. Dynamo's RAII guarantee carries no tenant identity.
- **Photon Extension:** Session-aware retention. Blocks belonging to a paused agent session, for example one waiting on a tool call, are protected from eviction until the session resumes or times out.
- **Rust Concepts:** `Drop`, `Arc` and `Weak`, interior mutability, lifetimes, and typestate for the block lifecycle.
- **Dynamo Reading:** `lib/kvbm-logical`, specifically `pools`, `blocks`, `manager`, and `tinylfu.rs`.
- **Proving Test:** No block is leaked or freed twice, and no handle crosses a tenant boundary.

Milestone 3: Radix Index

- **Problem:** Comparing a prompt against every cached sequence does not scale.
- **Build:** A tree in which each root-to-node path is a sequence of block hashes and each node is a cached block. A lookup descends while hashes match. Radix compression merges chains of single-child nodes into one node. The operations are insert, longest-prefix match, and eviction of least recently used leaves. The index is built single-threaded first, then made concurrent.
- **Photon Extension:** One index shared by the block pool and the router, so routing decisions use exact cache state rather than state synchronized by events. This is the split-KV-state necessity behind Photon's design.
- **Rust Concepts:** `Send` and `Sync`, locking strategy, and benchmarking with criterion.
- **Dynamo Reading:** `lib/kv-router/src/indexer`, including `concurrent_radix_tree_compressed`. The router uses this index to track which worker holds which blocks.
- **Proving Test:** Random operation sequences applied to the tree and to a brute-force list return identical results.

Milestone 4: Scheduler

- **Problem:** The GPU executes in steps. Each step decodes one token per running request and processes new prompts. A scheduler selects the contents of each step.
- **Build:**
  - **Continuous Batching:** Finished requests leave and waiting requests join after every step, rather than the batch waiting for its slowest request.
  - **Token Budget and Chunked Prefill:** Each step has a token cap. Long prompts are split across several steps so that requests in decode are not stalled.
  - **Preemption:** When KV memory is exhausted, a running request pauses. Its blocks are either copied to CPU memory (swap) or discarded and recomputed later (recompute).
  - **Targets:** TTFT and TPOT.
- **Gap Extensions:**
  - **CPU and GPU Overlap:** Plan step N+1 on the CPU while the mock executor runs step N, and measure scheduling overhead as a share of step time. Photon's first necessity is that this overhead caps GPU utilization on Blackwell-class decode steps of a few milliseconds.
  - **Swap Versus Recompute Cost Model:** Choose per request from block count, transfer bandwidth, and prefill cost, rather than by fixed policy.
  - **Speculative Decoding:** Model draft tokens and acceptance rates in the scheduler's token accounting.
- **Rust Concepts:** Async with tokio, channels, state machines, and pipelined execution.
- **Dynamo Reading:** `lib/mocker/src/scheduler`, and the speculative-decoding parameters in `lib/mocker/src/common/protocols.rs`.
- **Proving Test:** No step exceeds the token budget, no request waits indefinitely, and overlapped scheduling produces the same batches as sequential scheduling.

Milestone 5: KV Router

- **Problem:** Several engine instances (workers) serve traffic, and each new request needs a worker.
- **Build:** A scoring function that weighs cached-prefix overlap against current load. The worker holding the most of the prompt saves the most prefill but may already be overloaded.
- **Gap Extension:** A fail-closed eligibility filter applied before scoring. A request whose policy admits no worker is rejected rather than routed to the best-scoring worker. The Inference Serving Stack Primer identifies router eligibility (allowed worker IDs and required taints) as Dynamo's native fail-closed point.
- **Photon Extension:** Session affinity for multi-turn agent traffic, so later turns of a session route to the worker holding its KV.
- **Rust Concepts:** Trait objects versus generics, and plugin design.
- **Dynamo Reading:** `lib/kv-router/src/scheduling`, including `filter.rs`; `lib/kv-router/src/session_prefix_index.rs`; and `lib/llm/src/session_affinity`.
- **Proving Test:** Equal scores resolve to the same worker every time, and an empty eligible set never yields a routing decision.

Milestone 6: Tenant Keys

- **Problem:** Shared prefix caching leaks information. A faster first token reveals that another tenant sent the same prefix.
- **Build:** A per-tenant salt mixed into the first block's hash. Because hashes chain, every later hash changes, and tenants never share blocks. The salt uses a keyed hash, such as a Hash-based Message Authentication Code (HMAC) with a per-tenant secret derived from verified workload identity. A request without verified identity is rejected rather than served from the shared cache (fail closed).
- **Gap Addressed:** Dynamo's salt comes from the unverified `x-tenant-id` header, tenant identifiers are guessable and enable dictionary attacks on event-plane hashes, and the salt drops silently when request extensions are disabled or SGLang is used.
- **Rust Concepts:** Cryptography crates, error types, and API design.
- **Dynamo Reading:** `lib/kv-hashing/src/salt.rs`.
- **Proving Test:** Two tenants sending the same prompt receive zero cache hits on each other's blocks, and a request without verified identity receives no cache lookup.

Milestone 7: Disaggregated Handoff

- **Problem:** Prefill is compute-bound and decode is memory-bandwidth-bound. Dynamo can run them on separate workers and move KV blocks between them over the NVIDIA Inference Xfer Library (NIXL).
- **Build:** A prefill worker pool and a decode worker pool. A request is routed to prefill, its KV blocks are handed to a decode worker over a mock transport with configurable bandwidth and latency, and decode continues from those blocks. A conditional rule decides whether a request is worth disaggregating or should run on one worker.
- **Gap Extension:** Ownership of blocks in transit. The sending side holds each block until the receiver acknowledges it, and a failed transfer releases it. Dynamo's KVBM RAII guarantee stops at the process boundary.
- **Photon Context:** Disaggregation is designed in as a primitive rather than added through connectors, which is Photon's third necessity.
- **Rust Concepts:** Zero-copy buffers, async transport traits, and backpressure.
- **Dynamo Reading:** `lib/kv-router/src/conditional_disagg.rs` for the routing decision, `lib/kvbm-engine/src/leader/session` for transfer sessions and `BlockHolder`, and `lib/kvbm-physical/src/transfer` for block movement.
- **Proving Test:** A disaggregated run produces the same output tokens as an aggregated run, and a transfer killed midway leaks no blocks on either side.

Milestone 8: Discovery and Event Plane

- **Problem:** Workers join, leave, and fail. The router must know which workers exist and which blocks each one holds.
- **Build:** Workers register with a lease and renew it by heartbeat; an expired lease removes the worker. Workers publish block-stored and block-removed events, and the router consumes them to keep the Milestone 3 index current.
- **Gap Extension:** Authenticated KV events. Each event carries a message authentication code bound to the worker's identity, and the router discards unauthenticated events. Combined with Milestone 6 keyed hashes, an observer of the event plane can no longer infer shared-prefix relationships. Dynamo's ZMQ event plane carries no authentication or encryption.
- **Rust Concepts:** Async streams, cancellation, and backpressure.
- **Dynamo Reading:** `lib/runtime/src/discovery`, `lib/runtime/src/transports`, and the NATS authentication options in `nats.rs` for comparison.
- **Proving Test:** After a worker's lease expires or it reconnects, the router index matches actual worker state with no events lost or applied twice, and a forged event changes nothing.

Milestone 9: Request Migration

- **Problem:** A worker can fail mid-generation. The request should complete on another worker without the client noticing.
- **Build:** A retry manager that detects a failed stream, selects a new worker, replays the tokens already delivered, and resumes generation. Fault injection kills workers at controlled points.
- **Gap Extensions:**
  - **KV-Preserving Migration:** When the KV blocks survive, for example on a lower tier or a peer, transfer them to the new worker instead of recomputing prefill. Dynamo replays tokens and recomputes the KV cache.
  - **Deterministic Mode:** Seeded sampling and seeded fault injection, so a failed run can be replayed exactly. Dynamo offers no deterministic mode.
- **Rust Concepts:** Error propagation, idempotency, and deterministic fault injection.
- **Dynamo Reading:** `lib/llm/src/migration.rs`, including its open notes on replay capability for decoder and sampler state.
- **Proving Test:** A request whose worker is killed mid-generation completes on another worker with no duplicated or missing tokens, and a seeded run reproduces identical output.

Milestone 10 (Optional): Offload Tier

- **Problem:** GPU memory is small relative to reusable KV. Evicted blocks can move to CPU memory or disk instead of being discarded.
- **Build:** A second, larger, slower tier behind the Milestone 2 pool. Eviction demotes blocks in asynchronous batches, and a cache hit on the lower tier promotes them back with bulk reads, matching KVBM's throughput-bound profile.
- **Gap Extension:** Peer fetch, in which a worker pulls a block from another worker's lower tier through the index rather than by explicit point-to-point request. This also realizes global eviction across workers, part of Photon's split-KV-state necessity.
- **Rust Concepts:** Async I/O, memory-mapped files, and trait-based tier abstraction.
- **Dynamo Reading:** `lib/kvbm-physical`, including `layout`, `manager`, and `transfer`, and `lib/kvbm-engine`.
- **Proving Test:** Under a working set larger than the first tier, hit rate exceeds the single-tier baseline.

Dynamo Coverage

Covered by Milestones 1 Through 9

- **Hashing Contract:** How the router, KVBM, and the engines agree on block identity, including multimodal content and salt.
- **KVBM Logical Layer:** Block lifecycle, reference counting, reuse, and eviction.
- **KV-Aware Routing:** The index, the eligibility filter, session affinity, and the scoring decision.
- **Engine Scheduling Model:** Batching, chunked prefill, preemption, and speculative decoding as modeled in the mocker.
- **Disaggregated Serving:** The prefill and decode split and the block handoff.
- **Distributed Runtime:** Discovery, leases, and the event plane.
- **Resiliency:** Request migration under worker failure.

Not Covered

- **KVBM Physical Tiers:** Unless Milestone 10 is completed.
- **Planner:** Resizing prefill and decode pools to meet latency targets. Implemented in Python.
- **Engine Connectors:** Integration of vLLM, SGLang, and TensorRT-LLM. Implemented in Python.
- **Frontend and Kubernetes Operator.**
- **Governance:** Client authentication, authorization, attestation, and signed audit records, beyond what Milestones 5, 6, and 8 touch.

Contribution Leads

Each lead follows from a gap extension. None has been checked against existing issues or pull requests, and each changes behavior across components, so a design issue precedes code.

1. **Keyed Tenant Salt (Milestone 6):** `salt.rs` derives the salt with `xxh3_64_with_seed(value, 1)`, a fast non-cryptographic hash with no secret key, so anyone who knows or guesses the salt string can reproduce block hashes. A keyed hash changes the hashing contract across the router, KVBM, and engine producers.
2. **Authenticated ZMQ Events (Milestone 8):** NATS supports authentication; the ZMQ event plane does not, and its events expose block hashes.
3. **KV-Preserving Migration (Milestone 9):** Transfer surviving blocks instead of recomputing prefill. `migration.rs` already notes an undefined replay-capability contract for decoder and sampler state.
4. **Per-Tenant Cost Attribution (Observability Track):** Extend the request trace from capturing a tenant header to attributing token and cache cost.

The lower-risk first pull request remains tests or documentation in a crate studied during Milestones 1 through 3, or resiliency tests from Milestones 8 and 9.

Market and Business Context

NVIDIA Dynamo Position

Dynamo is an open-source (Apache 2.0) orchestration layer that sits above inference engines (vLLM, SGLang, and TensorRT-LLM) rather than replacing them. Version 1.0 reached general availability on March 16, 2026; containers are at 1.5.0. Adopters include Baseten, Deep Infra, Fireworks, and Together AI.

Dynamo assumes deployment inside a trusted network boundary. NVIDIA does not publish a rationale; the architecture and market suggest six drivers:

1. **Data Path Cost:** KV transfers over NIXL use Remote Direct Memory Access (RDMA), NVLink, and GPUDirect to bypass the kernel; per-transfer software encryption would sit on the TTFT critical path.
2. **Target Customer:** Hyperscalers, neoclouds, and inference providers operate single-owner clusters on physically isolated fabrics.
3. **High-Performance Computing (HPC) Lineage:** Dynamo inherits the posture of Triton and the NVIDIA Collective Communications Library (NCCL), which assume a trusted fabric.
4. **Kubernetes Convention:** Authentication and authorization are delegated to ingress, gateways, or service meshes.
5. **Velocity:** Components such as ZMQ, NATS, and etcd were built for trusted networks; hardening is arriving incrementally.
6. **Hardware Business Model:** NVIDIA addresses untrusted networks with BlueField Data Processing Units (DPUs), ConnectX crypto offload, and confidential computing.

The assumption conflicts with zero trust requirements in Office of Management and Budget (OMB) memorandum M-22-09 and National Institute of Standards and Technology (NIST) SP 800-207, which prohibit granting trust based on network location, and with environments where the fabric is not physically controlled. Milestones 6 and 8 build against this assumption.

Photon Intelligence

What It Is Building

Bryan Gopal's stealth company, met with on September 30, 2026, is the employer behind the recruithaus "Member of Technical Staff, Inference Systems" posting in Palo Alto. The company is building a new inference serving engine and the full stack above it, from scratch in Rust.

- **Engine:** A new runtime owning batching, scheduling, KV cache management, and prefix caching. It replaces vLLM, SGLang, and TensorRT-LLM rather than wrapping them.
- **Stack:** The same team owns request routing and multi-GPU, multi-node serving, which is the layer Dynamo occupies.
- **Priorities:** Latency, throughput, and cost per token.
- **Compensation:** $230K to $350K base.

Technical Necessity

1. **CPU Overhead Caps GPU Utilization:** Blackwell-class decode steps take a few milliseconds, and Python scheduling, garbage collection, and Global Interpreter Lock contention consume a growing share. Targeted by Milestone 4.
2. **Split KV State:** Engines own the KV cache while routing and offload run in a separate layer synchronized by events. A single runtime can route on exact state and evict globally. Targeted by Milestones 2, 3, and 10.
3. **Retrofitted Disaggregation:** vLLM and SGLang added prefill and decode separation through connectors; a clean-slate design makes it a primitive. Targeted by Milestone 7.
4. **Workload Shift:** Agentic, multi-turn, long-context traffic requires session-aware KV retention and pause and resume around tool calls. Targeted by Milestones 2 and 5.
5. **Rust Validation:** Dynamo's runtime already demonstrates Rust at this layer.

Feasibility

Kernels (FlashInfer, FlashAttention, CUTLASS, Triton, NCCL, NIXL) are open source and callable from Rust. The novel work is systems software, which a small team can deliver, as vLLM and SGLang demonstrated. Model definitions remain in PyTorch, and launch scope can be limited to the dominant model families.

Fundability

Company	Basis	Funding
Inferact	vLLM maintainers	$150M seed at $800M valuation, January 2026
RadixArk	SGLang spinout	$100M seed at $400M valuation, NVIDIA participating
Baseten	Inference provider	Reported $1.5B at $13B valuation, June 2026
Fireworks	Inference provider	Reported $17.5B valuation on $1B annual recurring revenue

Cost per token is the gross-margin lever for every inference provider, so engine gains convert directly into margin. A clean-slate Rust stack without backward-compatibility obligations is a distinct position from the Python-centric incumbents.

Funding Data Points

- **Public Record:** No funding is disclosed. The company website is a placeholder.
- **Recruiter Account:** Conversations with Amy Szymarek indicate a Series A of $10M to $20M.
- **Founder Account:** Bryan Gopal cited 100 months of runway.
- **Current Activity:** The company is raising a Series B.
- **Reconciliation:** 100 months on $10M to $20M implies burn of $100K to $200K per month, which supports 3 to 6 people at the posted salary band using GPU credits. The team implied by the posting (ten engineers plus compute) burns roughly $500K per month, yielding 20 to 40 months. The Series B would fund the hiring the posting describes.

Risk

The overall risk is high, falling to moderate if the Series B closes with a credible lead.

- **Funding:** The runway claim and the hiring plan do not reconcile without a larger raise or a smaller team.
- **Competition:** Inferact, RadixArk, and NVIDIA's free TensorRT-LLM and Dynamo stack are all moving their hot paths out of Python.
- **Technical:** Each new model architecture and hardware target requires support; incumbents absorb this with hundreds of contributors.
- **Go-To-Market:** Inference providers already run open engines with in-house teams. The business model (open source, license, or service) is undisclosed.
- **Visibility:** Stealth status leaves little to verify publicly.
- **Mitigants:** A small team can prove a benchmark advantage cheaply, and strong engine teams command an acquisition floor.

Potential Acquirers

Category	Examples	Rationale
Chip vendors	NVIDIA, AMD	Strongest category; NVIDIA acquired Deci, OctoAI, and Run:ai; AMD acquired Nod.ai, Brium, and the Untether AI team
Neoclouds	CoreWeave, Nebius, Lambda, Crusoe	Moving up the stack from GPU rental
Inference providers	Baseten, Fireworks, Together	Own the performance edge rather than license it
Hyperscalers	AWS, Google Cloud, Azure, Oracle	Team acquisitions
Enterprise platforms	Red Hat, Databricks	Red Hat acquired Neural Magic, a core vLLM contributor

Inference providers are the weakest buyers but plausible acquirers.

Embedded LLM, WEKA, and Neocloud 2.0

Embedded LLM

Embedded LLM is a Singapore company founded in 2023 with approximately 41 employees and a seed round from 500 Global in October 2025. Its products are TokenVisor (a control plane for AMD GPU neoclouds covering pricing, metering, rate limiting, billing, and governance), TokenVisor Spaces (persistent agent workspaces on AMD EPYC and Instinct hardware), and JamAI Base. Its serving layer is vLLM. It is too small to acquire Photon and is better viewed as a channel into AMD-powered clouds.

Neocloud 2.0

The term is not an industry standard; it is the stated mandate of an Embedded LLM board member appointed in September 2026: "Creating the Neocloud 2.0 operating system."

Dimension	Neocloud 1.0	Neocloud 2.0
Product	GPU-hours and bare metal	Tokens, managed inference, agent runtimes
Margin	55% to 65% gross, eroding as supply normalizes	Software-like margin on the same hardware
Switching cost	Low	Higher, through APIs, billing, and workflows

The operating system comprises five layers: Fleet management, serving, a multi-tenant control plane, an agent runtime, and a developer surface. AMD fleets lack the integrated stack NVIDIA provides through Dynamo and NVIDIA Inference Microservices (NIM), which defines the market opening. McKinsey identifies the durable neocloud paths as sovereign or regulated niches, loyalty from AI startups, or consolidation.

WEKA Fit

The board appointment links WEKA to Embedded LLM. WEKA supplies the memory layer: Augmented Memory Grid offloads KV cache to a persistent NVMe "Token Warehouse" over RDMA and GPUDirect Storage, integrates with TensorRT-LLM and LMCache through an open-source NIXL plugin, and works with Dynamo. WEKA claims 1000x the KV capacity of DRAM and up to 41x faster TTFT, with deployments on Nebius and Oracle Cloud. Milestone 10 models this layer.

Layer	Owner
Control plane	Embedded LLM (TokenVisor)
Agent runtime	Embedded LLM (TokenVisor Spaces)
Serving engine	vLLM
KV memory tier	WEKA (Augmented Memory Grid)
Storage	WEKA (NeuralMesh)

The fit is tightest at the agent runtime, where persistent sessions require KV state that survives pauses and tool calls. WEKA is the more likely strategic investor or acquirer of Embedded LLM. The principal gap is that Augmented Memory Grid is built on NVIDIA GPUDirect Storage, with no stated AMD GPU support.

Open Questions

1. **Photon Series B:** Size, valuation, lead investor, and term sheet status.
2. **Photon Traction:** The milestone justifying a Series B for a company building from scratch.
3. **Photon History:** Founding date, uses of prior rounds, and any product change.
4. **Photon Runway:** How the 100-month figure is calculated and the 18-month headcount plan.
5. **Photon Model:** Open source, license, or service; benchmark target; launch model and hardware scope.
6. **WEKA On AMD:** Whether Augmented Memory Grid will support AMD Instinct.

Interview Preparation

Format

The interview is conducted in Rust or Go, with AI coding tools allowed. Weight therefore shifts from writing syntax toward decomposition, correctness, tests, and review of generated code.

Practice Protocol

- **Timebox:** Each drill runs 45 to 60 minutes, using the AI tool intended for the interview, with reasoning stated aloud.
- **Tests First:** Tests are written before generated code is accepted. They catch AI errors live and demonstrate judgment.
- **Rust Review:** Generated Rust often compiles only after added clones, `Arc<Mutex<_>>` wrappers, or lifetime workarounds that conceal a poor ownership design. Detecting and correcting these in review is the Rust depth the project builds.
- **Photon Framing:** The Milestone 3 shared index, the Milestone 4 overlap extension, and the Milestone 7 handoff map directly to Photon's split KV state, CPU overhead, and disaggregation necessities, and are the strongest discussion material for its engine-internals focus.

Language Choice

- **Rust:** Matches Photon's runtime. Borrow-checker and concurrency errors cost time under a timebox.
- **Go:** The stronger production language from prior work and the lower-risk choice for speed.
- **Decision Rule:** Rust if Milestones 1 through 3 are complete before the interview. Otherwise Go, with the Rust project presented as evidence of direction.

The project remains in Rust regardless, because Goals 1 and 2 depend on it.

Environment

- **Milestones 1 Through 10:** CPU-only, on a MacBook Pro M1 Max (Jeeves7).
- **Real Forward Pass:** A later extension that calls FlashInfer from Rust through the foreign function interface (FFI) would run on a DGX Spark. That effort is not scoped.

Sources

- Inference Serving Stack Primer: NVIDIA Dynamo, Photon Intelligence, and Neocloud 2.0 (project document)
- NVIDIA Dynamo Distinguished Engineer Posting (JR2009883): LinkedIn job listing, provided as a PDF
- NVIDIA Dynamo Repository: https://github.com/ai-dynamo/dynamo
- Dynamo `kv-hashing` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kv-hashing
- Dynamo `kv-router` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kv-router
- Dynamo `kvbm-logical` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kvbm-logical
- Dynamo `kvbm-physical` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kvbm-physical
- Dynamo `kvbm-engine` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/kvbm-engine
- Dynamo `mocker` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/mocker

- Dynamo `runtime` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/runtime
- Dynamo `tokens` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/tokens
- Dynamo `llm` Crate: https://github.com/ai-dynamo/dynamo/tree/main/lib/llm
- Dynamo KVBM Design: https://docs.nvidia.com/dynamo/design-docs/kvbm-design
- Dynamo Mocker Guide: https://docs.nvidia.com/dynamo/user-guides/testing/mocker
- Dynamo Request Migration: https://docs.nvidia.com/dynamo/dev/kubernetes/fault-tolerance/request-migration
- Dynamo Router Filtering: https://docs.nvidia.com/dynamo/dev/knowledge-base/modular-components/router/router-filtering
- Dynamo Secure Deployment Guidelines: https://docs.nvidia.com/dynamo/dev/security/secure-deployment-guidelines
- Dynamo 1.3 Request Extensions and Cache Salt: https://docs.nvidia.com/dynamo/v1.3.0/additional-resources/nvidia-request-extensions-nvext
- NVIDIA Dynamo 1.0 Press Release: https://investor.nvidia.com/news/press-release-details/2026/NVIDIA-Enters-Production-With-Dynamo-the-Broadly-Adopted-Inference-Operating-System-for-AI-Factories/default.aspx
- Dynamo Roadmap, Releases 1.2 to 1.4: https://github.com/ai-dynamo/dynamo/issues/9178
- Dynamo v1.4.0 Release Notes: https://docs.nvidia.com/dynamo/reference/releases/v1-4-0
- IBM Redbooks KV Cache Platform: https://www.redbooks.ibm.com/docs/MD260021/MD260021.html
- VAST Data on Dynamo KV Offload: https://www.vastdata.com/blog/nvidia-dynamo-vast-scalable-optimized-inference
- Inferact Seed Financing (Bloomberg): https://www.bloomberg.com/news/articles/2026-01-22/andreessen-backed-inferact-raises-150-million-in-seed-round
- RadixArk Seed Financing: https://techfundingnews.com/radixark-100m-seed-accel-spark-nvidia-sglang-ai-inference/
- Baseten Funding (TechCrunch): https://techcrunch.com/2026/06/18/ai-inference-startup-baseten-reportedly-raising-1-5b-months-after-its-last-mega-round/
- Fireworks Valuation: https://valueaddvc.com/blog/fireworks-ai-valuation-2026-17-5b-series-d-and-the-1b-arr-inference-business
- Photon Intelligence Website: https://www.photonintelligence.com/
- Embedded LLM Profile (PitchBook): https://pitchbook.com/profiles/company/543318-58
- TokenVisor Spaces Launch: https://finance.yahoo.com/technology/ai/articles/agentic-ai-runs-where-enterprise-185200063.html
- TokenVisor for AMD Neoclouds: https://securitybrief.asia/story/tokenvisor-aims-to-optimise-monetisation-of-amd-gpu-llm-neoclouds
- McKinsey on Neocloud Evolution: https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-evolution-of-neoclouds-and-their-next-moves
- WEKA Augmented Memory Grid: https://www.weka.io/product/augmented-memory-grid/
- WEKA on Nebius: https://www.weka.io/article/benchmarking-weka-on-nebius-ai-cloud
- WEKA on Oracle Cloud: https://blogs.oracle.com/ai-and-datascience/scaling-long-context-inference-on-oci-with-wekas-augmented-memory-grid

Acronyms

Acronym	Expansion
CPU	Central Processing Unit
DPU	Data Processing Unit
FFI	Foreign Function Interface
GPU	Graphics Processing Unit
HMAC	Hash-based Message Authentication Code
HPC	High-Performance Computing
ITL	Inter-Token Latency
KV	Key-Value
KVBM	KV Block Manager
LLM	Large Language Model
LRU	Least Recently Used
NCCL	NVIDIA Collective Communications Library
NIM	NVIDIA Inference Microservices
NIST	National Institute of Standards and Technology
NIXL	NVIDIA Inference Xfer Library
NVMe	Non-Volatile Memory Express
OMB	Office of Management and Budget
RAII	Resource Acquisition Is Initialization
RDMA	Remote Direct Memory Access
TLS	Transport Layer Security
TPOT	Time Per Output Token
TTFT	Time to First Token
ZMQ	ZeroMQ